

Tracker & Scanner Build

Two boards, wired and powered from the ground up: the wearable tracker a child carries, and the RFID scanner bolted at the school gate. Same rule breaks more builds than any bug does — get the power wiring right first.

Shared ground rules · The tracker · The scanner · How they talk to each other

Before wiring anything

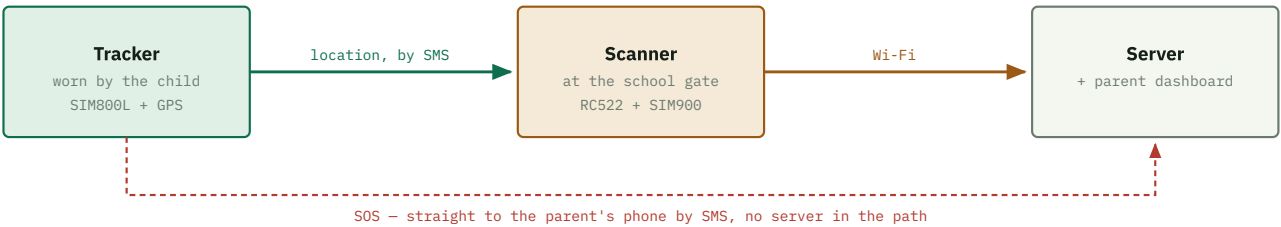
Both boards carry a 2G modem, and both modems share the same trap: they pull far more current than they idle at, in short bursts, whenever they transmit. Everything below follows from that one fact.

THE ONE RULE THAT MATTERS MOST

- A GSM modem can pull up to **2 A** for a fraction of a second while transmitting, at **3.4–4.4 V**. Neither board's onboard 3.3 V regulator nor a USB port can supply that — give every modem its own direct feed at that voltage.
- Put a **1000–2200 μ F electrolytic capacitor** across the modem's power pins, physically as close to the module as the wiring allows. Without it, the transmit burst dips the rail far enough to reset the whole board — which looks exactly like a firmware bug and wastes a weekend proving it isn't.
- Use thick, short power wires to the modem. A thin jumper has enough resistance on its own to cause the same brownout.
- The RFID reader and the GPS module are both **3.3 V only** — 5 V destroys either one outright.

Bring a board up one part at a time — power the MCU alone first, then add one module, prove it works over serial, then add the next. Wiring everything at once and flashing the full firmware turns every fault into "nothing works," with no way to tell which part is actually broken. Both build orders below follow that same discipline.

The system at a glance



Each device is a separate physical build with its own power supply.
They only meet over SMS and Wi-Fi – nothing is wired between them.

DEVICE 1

The tracker

Worn by the child. One ESP32, a 2G modem for SMS, a GPS module, and a recessed SOS button – all of it running off a single LiPo cell.

A

Parts

PART	ROLE	NOTE
ESP32 DevKit v1 (WROOM-32)	Main board	Its built-in Wi-Fi and BLE scanning double as two of the four position sources
SIM800L	2G / GSM modem	SMS is the whole uplink in this build – see power rules
NEO-6M	GPS module	Keep its backup pin powered or every fix restarts cold
Momentary push button	SOS trigger	Recessed, on a wake-capable pin
LiPo cell, 2500–3000 mAh	Battery	Feeds the modem directly, not through the regulator
TP4056 module	Charger	Charges the battery only – powers nothing else
Electrolytic capacitor, 1000–2200 µF	Bulk cap	At the modem's power pins. Not optional
LED + 220 Ω resistor	Child-facing feedback	Standing in for a vibration motor that's out of stock

PART	ROLE	NOTE
Passive piezo buzzer	Extra feedback	Same GPIO family, driven with <code>tone()</code>
Resistors, 1 k Ω + 2.2 k Ω	Level shifter	Protects the modem's input pin from 3.3 V logic it isn't rated for

B How it's powered

The battery is the supply, not a backup. USB only ever reaches the TP4056, which charges the LiPo and nothing else. The LiPo's raw 3.7–4.2 V then goes **straight to the SIM800L**, because the modem's transmit bursts need far more current than the ESP32's onboard 3.3 V regulator can give. That same 3.3 V rail is fine for the ESP32 and the GPS module, which draw much less. Every ground is tied together across the whole board.

C Wiring

SIGNAL	GPIO	NOTE
SIM800L TX → ESP32 RX	16	UART2
ESP32 TX → SIM800L RX	17	Through the 1 k Ω / 2.2 k Ω divider, $\approx$ 2.3 V at the modem — its RX pin isn't 3.3 V tolerant. The other direction needs no shifting; the modem's $\approx$ 2.8 V output already reads as logic HIGH
SIM800L PWRKEY	23	Pulse LOW $\approx$ 1 s to toggle the modem on or off
SIM800L power gate	32	High-side MOSFET, cuts power to the modem entirely
SIM800L DTR	19	Held LOW to wake the modem from its sleep mode
NEO-6M TX → ESP32 RX	25	UART1, remapped off its default pins
ESP32 TX → NEO-6M RX	26	UART1, remapped
NEO-6M power gate	27	MOSFET — but the module's backup-power pin (V_BCKP) stays powered regardless of this gate. Cutting it turns every fix into a slow cold start instead of a fast one, which costs more energy than the gating saves
SOS button	33	One of the few pins that can wake the board from deep sleep. Active low, internal pull-up
Feedback LED	13	Through a 220 Ω resistor to ground

SIGNAL	GPIO	NOTE
Piezo buzzer	12	Passive, driven with <code>tone()</code> . This pin also sets flash voltage at boot — don't repurpose it casually
Onboard debug LED	2	Not the child-facing one
Battery sense	34	Has to be a pin in the 32–39 range — the other analog input can't be read while Wi-Fi is on. 100k/100k divider

D Build it in this order

1	ESP32 alone Blink the onboard LED, blink the feedback LED, sound the piezo. Confirms the board, the supply, and the feedback wiring before anything else can confuse the picture.
2	GPS on its own Power it from 3.3 V and watch its serial output. Expect GPS sentences within a minute outdoors, up to five indoors. Nothing coming through usually means TX/RX are swapped, or the module was run at 5 V and is dead.
3	Modem on its own supply Power it from the LiPo, before any code runs. Confirm its status LED is blinking, then confirm it answers a basic command, shows it's registered on the network, and can send one text by hand.
4	SOS button Confirm the pin reads low while held and high the rest of the time.
5	Full firmware Only now. Everything gets wired together and flashed once every part has already proven itself alone.

E If something's wrong

SYMPTOM	USUAL CAUSE
Board resets whenever a text sends	Missing or undersized bulk cap on the modem's power rail — the single most common fault on this build
Modem never answers a basic command	TX and RX swapped — costs nothing to just try them the other way
Modem answers, but replies are garbled	Baud mismatch, or the level shifter on GPIO 17 is missing

SYMPTOM	USUAL CAUSE
Modem stays stuck "searching" for the network	No 2G coverage, the SIM isn't registered to an identity, or its prepaid load has expired
GPS takes ≈ 27 s for every single fix	Backup power (V_BCKP) isn't reaching the module between power cycles
GPS produces nothing at all	TX/RX swapped, or the module was run at 5 V
Button doesn't wake the board from sleep	Wired to a pin that isn't RTC-capable
Works fine on USB, fails on battery	The battery sags under the modem's transmit burst — try a fresher cell and thicker wire

DEVICE 2 The scanner

Bolted at the gate, mains-powered. A card tap reads the tag, checks it against a cached list, and both logs it to the network and can text the parent directly — either path works without the other.

A Parts

PART	ROLE	NOTE
ESP32 DevKit v1	Main board	Mains powered, so battery life isn't a constraint here
MFRC522 (RC522)	RFID reader	13.56 MHz. 3.3 V only — 5 V destroys it
DS1302	Real-time clock	Three-wire, not I ² C. Coin-cell backed, so it survives outages and reboots
SIM900	2G / GSM modem	A second, independent uplink — texts the parent even when the Wi-Fi is down
16×2 character LCD, I ² C backpack	Display	Optional, but turns a beep into a name on screen
Buzzer	Audio feedback	Driven through a transistor — never straight off a GPIO pin

PART	ROLE	NOTE
Green + red LED, or one 4-pin RGB LED	Status lights	Either works — an RGB LED's red/green legs land on the same two pins the discrete LEDs would use
5 V USB power supply	Power	Into the board's VIN pin, never into the RC522

B How it's powered

The SIM900 has the same appetite as the tracker's modem — up to 2 A in bursts at 3.4–4.4 V — and the same rule applies: it can't run from the ESP32's 3.3 V pin or straight off USB 5 V. Give it its own regulator stepped down to roughly 4 V (or a shield that already has one), plus the same 1000–2200 μ F bulk cap right at its power pins. Being on mains power removes the battery-budget question here, not the current spike.

C Wiring

SIGNAL	GPIO	NOTE
RC522 SDA / SS	5	SPI
RC522 SCK	18	SPI
RC522 MOSI	23	SPI
RC522 MISO	19	SPI
RC522 RST	4	Moved off its usual pin to keep I ² C free
RC522 power	—	3.3 V only, never 5 V
DS1302 CLK	14	Three-wire interface
DS1302 DAT	15	Three-wire
DS1302 CE	2	Three-wire
LCD SDA	21	I ² C — the only thing on this bus
LCD SCL	22	I ² C
SIM900 TX → ESP32 RX	16	UART2
ESP32 TX → SIM900 RX	17	UART2

SIGNAL	GPIO	NOTE
SIM900 PWRKEY	32	See the caution below — this one bites
Buzzer	25	Through a transistor
LED, green / OK	26	Green leg, if using an RGB LED
LED, red / error	27	Red leg, if using an RGB LED
LED, blue (RGB only)	13	Ignored unless the RGB option is wired in. Chosen because it carries no boot-time role, unlike pin 12

THE SIM900'S POWER BUTTON ISN'T A POWER BUTTON

PWRKEY is a **toggle**, not an on switch — every ≈ 1.2 s LOW pulse flips the modem between on and off. That matters because the ESP32's own boot leaves GPIO 32 floating for a moment, and if it floats low, the modem's power state flips at random on every reset. Wire it exactly as specced, and let the firmware pin the line high the instant it starts — that's what makes the boot behavior predictable instead of a coin flip.

GET THE LED WIRING TYPE RIGHT

If you use a single 4-pin RGB LED instead of two discrete ones, its wiring type has to match what the firmware expects. Common-cathode (the shared leg to ground, lit on HIGH) is the default assumption. Wire a common-anode LED without changing that setting and every cue inverts — on at idle, dark during an actual alert.

WHY THE CLOCK ISN'T OPTIONAL

The ESP32 has no clock of its own; it only knows the time by asking the network, and a school network will drop. The DS1302 keeps ticking through outages and reboots on its coin cell, so a tap that gets buffered during an outage still carries the moment it actually happened, not a guess made later when the connection comes back.

D Build it in this order

1 ESP32 alone

Confirm the board boots and the LEDs and buzzer respond to a basic test.

2 RC522

Tap a known card and confirm its ID prints over serial. Wrong readings, or nothing at all, almost always means it was run at 5 V.

3 DS1302

Set the time once, power-cycle the board, and confirm the clock kept ticking on the coin cell rather than resetting.

4 LCD

Confirm it shows text over I²C at address 0x27.

5 SIM900

Power it and let the boot sequence run — the firmware probes it with a basic command before deciding whether to power-cycle it. Confirm it registers on the network and can send a text. It's the same command set as the tracker's modem, so the same checks apply.

6 Full firmware

Only now. Everything wired together and flashed once every part has proven itself alone.

E If something's wrong

SYMPTOM	USUAL CAUSE
Reader returns nothing, or a garbage ID	Powered at 5 V instead of 3.3 V — likely dead
Clock resets to a default time after every power cycle	Coin cell missing or dead, or CLK/DAT/CE wired out of order
LCD is blank or shows garbled characters	Wrong I ² C address, or SDA/SCL swapped
Status LEDs sit lit at idle, go dark during an alert	Common-anode LED wired without flipping the matching setting
Modem's power state flips unpredictably across resets	Expected if PWRKEY isn't pinned high fast enough at boot — see the caution above

A card can also carry a backup contact directly on itself, for when the network truly can't be reached. That data sits behind a key that's deliberately not the factory default, so writing new cards means using the project's own card-writer tool with the matching key.

How they talk to each other

The two boards never share a wire. The tracker's location reports travel to the scanner purely as SMS, addressed to the scanner's SIM900 number; the scanner relays whatever it receives onward over its own Wi-Fi. An SOS from the tracker skips that whole path — it goes straight to the parent's phone by SMS, with no server and no scanner between them, so it survives either device being offline.

Pin maps are authoritative in `tracker/include/pins.h` and `scanner/include/pins.h` — if a pin ever changes, change it there first and this page second.

Build: `pio run -t upload -e tracker` · `pio run -t upload -e scanner`